

Yangguang: Vision-Driven NPCs Through Composed Language Models

Yibin Liang

School of International Innovation Creative Design
Shanghai University of Engineering Science
`Leido@sues.edu.cn`

January 2026

Abstract

This paper presents an architecture that combines Vision-Language Models (VLMs) and Large Language Models (LLMs) to create embodied AI agents in 3D virtual environments, implemented in Unity3D. The core question is: how can we enable text-based interaction grounded in 3D scenes on real-time rendering platforms? Rather than scripting behaviors, we give NPCs genuine autonomy to explore their surroundings. We detail the prompt engineering strategies that proved essential for coherent agent behavior, including perception prompts, decision prompts, and output format constraints. The resulting framework is modular and extensible—we hope it serves as a foundation for developers to build more interesting applications.

1 Introduction

Traditional game AI relies on fixed scripts and behavior trees [1]. I would argue these do not deserve to be called “AI” at all. Once you change the scene, their behavior becomes completely predictable, and the game loses what makes play interesting: improvisation, surprise, the unknown.

My first attempt used an LLM combined with raycasting, letting the NPC read objects directly from the Unity hierarchy. This approach had serious problems—the agent would get stuck on walls, walk in circles, and generally behave in ways that felt artificial. Any human player could immediately tell the NPC was “acting.” It lacked genuine understanding of its environment.

I then tried ML-Agents to address the wall-collision problem. This worked for navigation, but it still could not solve the fundamental issue: the agent did not actually *see* the world. Compared to traditional game AI, ML-Agents just adds a few extra steps to avoid certain problems—it is not true perception. This was the opposite of what I wanted.

Recent vision-language models (VLMs) [2] and large language models (LLMs) [3] offer a different path. Instead of hand-coding responses to every possible situation, we can let agents perceive the world directly and reason about what to do. The idea is simple: give an NPC a camera, describe what it sees using a VLM, and let an LLM decide how to act.

The result is an agent whose behavior emerges from perception rather than prescription. During experiments, something unexpected happened: the NPC walked to a window and stood there for a long time, gazing at the skybox outside. When asked what it was doing, it described the view in surprisingly poetic language. I had never programmed this behavior—it emerged from the agent’s own perception and reasoning. A video demonstration is available at the link in Section 9.

This paper describes how I built this system and what I learned. Section 3 covers the architecture, Section 4 the main components, Section 5 the prompt engineering strategies that proved critical, Section 6 the data flow, and Section 7 reports performance measurements. Section 8 discusses limitations.

2 Related Work

ML-Agents. Unity’s ML-Agents package is mature and widely used for training NPCs through reinforcement learning. I initially considered this approach, but found it problematic in terms of both time and hardware requirements. More fundamentally, ML-Agents addresses symptoms rather than causes: it can solve wall-collision issues, but it cannot give the agent genuine visual perception. The agent still operates on abstract state representations, not on what it actually “sees.” This contradicted my original vision.

Large Language Models. LLMs have been central to my work—they serve as the decision-making brain. The model itself is capable, but it lacks perceptual input. It can process information and reason, but without appropriate sensory data, there is nothing meaningful to process. The challenge is designing prompts and pipelines that provide the LLM with grounded, perceptual information.

Vision-Language-Action Models. Recent VLA models like PaLM-E [4] and OpenVLA [5] learn end-to-end mappings from pixels to actions, achieving impressive results in robotics. However, they require massive training datasets and offer limited interpretability. My approach takes the opposite path: compose off-the-shelf VLMs and LLMs at inference time, trading some potential performance for transparency and ease of iteration. When an agent makes a strange decision, I can read its reasoning in plain text.

3 System Architecture

The system runs as three layers that communicate through well-defined interfaces:

Runtime Layer (Unity). The agent exists here as a `GameObject` with a first-person camera and a `NavMeshAgent` for movement. A controller script called `LLMBrain` orchestrates the perception-action loop.

Network Layer. Two API clients handle external communication: `VLMClient` sends screenshots to Qwen-VL-Plus [6] and receives scene descriptions; `ExternalLLM` sends these descriptions to DeepSeek-V2 [7] for action decisions.

Cognitive Core. This is not a component but an emergent property—the “intelligence” that arises when visual perception feeds into language-based reasoning. I find this framing useful for thinking about where capabilities come from and where they break down.

4 Core Components

4.1 YangguangRealVision: Perception

Every few seconds (configurable, default 5s), this module captures a 512×512 screenshot from the agent’s first-person camera. The image is encoded as Base64 and sent to the VLM along with a carefully designed prompt (see Section 5.1). A typical response might be: “Grassy hillside ahead, large boulder to the left, forest edge visible in the distance.”

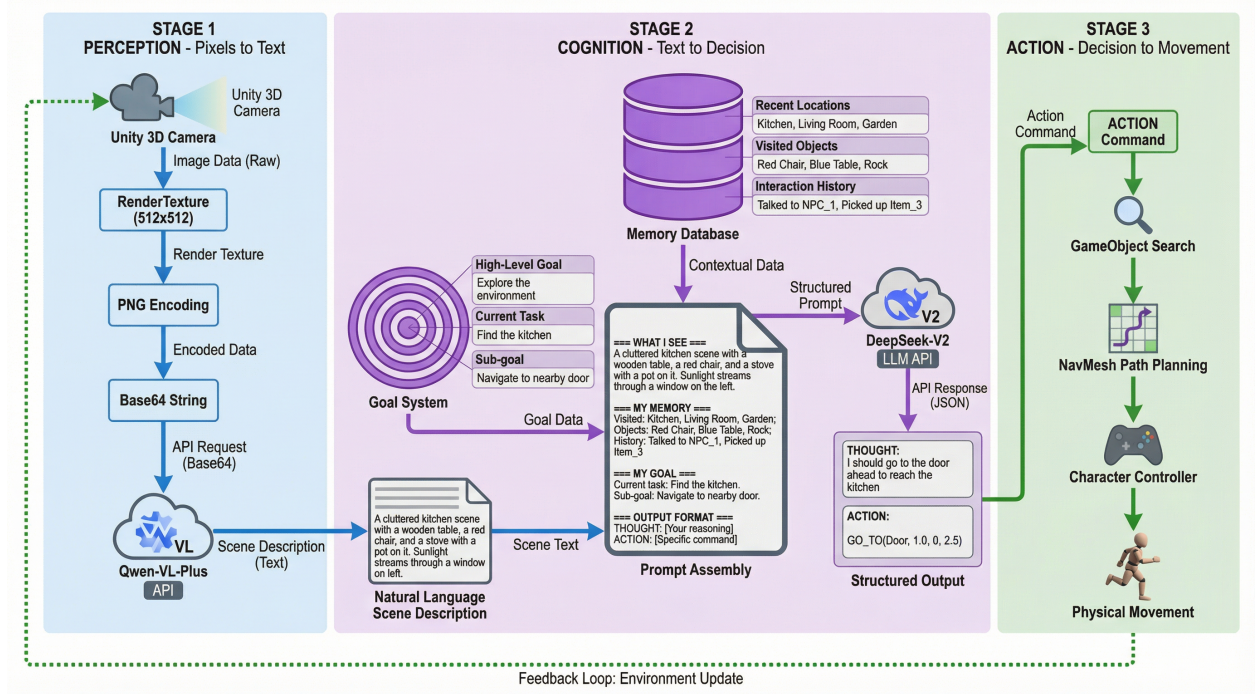


Figure 1: System architecture of Yangguang. Stage 1 (Perception) captures camera images and converts them to natural language via VLM. Stage 2 (Cognition) assembles contextual data from memory and goals, then queries the LLM for decisions. Stage 3 (Action) executes navigation commands through Unity’s NavMesh system. The feedback loop continuously updates the agent’s perception of its environment.

4.2 LLMBrain: Decision-Making

This component runs an Observe-Think-Act loop:

1. **Observe:** Receive the scene description from the perception module.
2. **Think:** Construct a prompt containing the current view, memory context, goal hierarchy, and output format specification (see Section 5.2).
3. **Act:** Parse the LLM’s response and dispatch a navigation command.

4.3 YangguangAI: Action Execution

This module translates high-level commands into NavMesh operations. For **GO_TO**, it searches the scene for matching objects and sets a destination. For **EXPLORE**, it selects a random reachable point within a specified radius. The abstraction is deliberately simple—complex motor control is not the focus here.

5 Prompt Engineering

Prompt design turned out to be the most critical and time-consuming part of this project. Small changes in wording could dramatically alter agent behavior—sometimes for the better, sometimes catastrophically. This section documents the strategies that worked.

5.1 VLM Perception Prompt

The prompt sent to the VLM with each screenshot went through many iterations:

Version 1 (Failed): “Describe this image in detail.”

This produced verbose, meandering paragraphs that overwhelmed the decision-making LLM. The agent would fixate on irrelevant details (“the texture of the wooden floor appears to be oak”) while missing obvious navigation cues.

Version 2 (Failed): “List all objects visible in this image.”

This produced dry inventories with no spatial relationships. The agent knew there was a door and a table, but not that the door was behind the table.

Version 3 (Working): “You are an explorer seeing this scene for the first time. Briefly describe what catches your attention and where things are relative to you. Focus on what matters for navigation and interaction.”

This framing produced descriptions that balanced informativeness with concision. The “explorer” persona encouraged the VLM to prioritize salient features, while “relative to you” grounded descriptions in egocentric spatial terms the LLM could act on.

5.2 LLM Decision Prompt

The decision prompt assembles multiple information sources into a structured format:

```
=== WHAT I SEE ===  
{VLM description}
```

```
=== MY MEMORY ===  
Recent locations: {last 5 places visited}
```

```

Objects encountered: {list}
Recent actions: {last 3 actions taken}

=== MY GOAL ===
High-level: {e.g., "Explore the environment"}
Current task: {e.g., "Find the kitchen"}
Sub-goal: {e.g., "Navigate toward the door ahead"}

=== OUTPUT FORMAT ===
You must respond in exactly this format:
THOUGHT: [Your reasoning about what to do next]
ACTION: [One of: GO_TO(object), EXPLORE, WAIT]

```

Several design choices proved important:

Memory section prevents loops. Without recent action history, agents would repeatedly visit the same locations. Including “I was just at the boulder” in context causes the LLM to naturally avoid repetition.

Goal hierarchy provides direction. Early versions had no goal structure, resulting in aimless wandering. The three-level hierarchy (high-level goal, current task, sub-goal) gives the agent purpose while allowing flexibility in execution.

Strict output format enables parsing. Free-form responses were impossible to reliably parse. The rigid THOUGHT/ACTION format, combined with a whitelist of valid actions, ensures the system can always extract executable commands. Invalid responses trigger a re-query.

5.3 Prompt Iteration Lessons

Some general lessons from prompt development:

Personas matter. Telling the VLM it is an “explorer” or the LLM it is a “curious wanderer” measurably changes output style. These framings are not just flavor—they bias the model toward certain response patterns.

Examples help but can overfit. Few-shot examples in the prompt improved consistency but sometimes caused the agent to mimic example behaviors too literally. I found one or two examples optimal; more caused overfitting.

Negative instructions often backfire. “Do not describe irrelevant details” paradoxically increased irrelevant details. Positive framing (“focus on navigation-relevant features”) worked better.

Temperature affects personality. Lower temperature (0.3-0.5) produced more consistent but “boring” agents. Higher temperature (0.7-0.9) created more varied behavior but occasional nonsense. I settled on 0.6 as a compromise.

6 Data Flow

6.1 Autonomous Behavior

The default loop runs continuously:

1. **t=0s:** Capture screenshot, send to VLM with perception prompt. (2–5s latency)

2. **t=5s:** Receive description, assemble decision prompt with memory and goals, send to LLM. (1–3s latency)
3. **t=8s:** Parse response, execute action, update memory.

Total cycle time is 3–8 seconds depending on API response times. The agent continues executing its previous action while waiting for new instructions.

6.2 Player Interaction

When a player speaks to the agent, their input is injected into the decision prompt as an additional section:

```
=== PLAYER SAID ===
"{player's message}"
Consider responding or adjusting your behavior.
```

The agent can then respond contextually—commenting on what it sees, answering questions about the environment, or adjusting its behavior based on requests.

7 Evaluation

7.1 Emergent Behaviors

The following behaviors were never explicitly programmed, but emerged consistently during testing:

Curiosity. Agents generate thoughts like “I see a hill I haven’t explored” and navigate toward it. This emerges from the LLM’s tendency to prefer novelty when given choices.

Aesthetic appreciation. As mentioned in the introduction, the agent sometimes stops at windows or scenic viewpoints and produces poetic descriptions of what it sees. This was entirely unexpected.

Memory-based avoidance. Including recent actions in the prompt causes agents to avoid repetitive loops: “I was just at that boulder, I should try somewhere else.”

Terrain inference. The LLM sometimes infers environmental properties from visual descriptions (“the ground looks uneven”) and adjusts behavior accordingly, even without explicit terrain data.

These behaviors are fragile—they depend heavily on prompt wording and can disappear with small changes. But they suggest that perception-grounded reasoning enables capabilities that would be tedious to hand-code.

7.2 Latency and Cost

At current API pricing, running one agent costs roughly \$0.86/hour—acceptable for research but too expensive for production games. Local model deployment would address both latency and cost.

8 Limitations and Future Work

To be clear about what the current system cannot do:

Real-time response. A 3–8 second action loop works for slow-paced exploration but is unusable for combat or time-sensitive gameplay.

Stage	Latency
Image capture and encode	approximately 60ms
VLM API call	2–5s
LLM API call	1–3s
Navigation dispatch	approximately 5ms
Total cycle	3–8s

Table 1: Latency breakdown for one perception-action cycle. API calls dominate; local deployment would substantially reduce total time.

Spatial memory. The agent has no persistent map. It remembers recent actions through the prompt but cannot build a mental model of the environment.

Fine motor control. All actions are high-level navigation commands. The agent cannot pick up objects, open doors, or perform precise movements.

Prompt brittleness. As discussed in Section 5, behavior is sensitive to prompt wording. This makes the system difficult to tune and potentially unstable across model updates.

Future directions include: local model deployment to reduce latency, vector databases for long-term spatial memory, VLA model integration for finer motor control, and more systematic prompt optimization methods. Multi-agent scenarios are also worth exploring—what happens when two perception-driven NPCs encounter each other?

9 Conclusion

This work demonstrates that composing existing VLMs and LLMs can produce NPC behaviors that feel qualitatively different from scripted AI. The agents exhibit apparent curiosity, avoid repetition, and respond to visual context in ways I did not anticipate.

The key technical contribution is not the architecture itself—which is straightforward—but the prompt engineering strategies that make it work. The perception prompt’s “explorer” framing, the decision prompt’s memory and goal structure, and the strict output format are all essential. Without these, the same architecture produces incoherent behavior.

I release this implementation hoping others will find it useful for exploring perception-grounded game AI.

Video Demonstrations:

- First working prototype: <https://youtu.be/OWpcWLnyKGc>
- Early version with wall-collision issues (white-box test): <https://youtu.be/DceI-MnvLxM>
- Final version showing NPC gazing out the window: <https://youtu.be/dcj-9YucPV4>

Source code: [GitHub link to be added]

References

- [1] Yannakakis, G. N., & Togelius, J. (2018). *Artificial Intelligence and Games*. Springer.
- [2] Radford, A., et al. (2021). Learning Transferable Visual Models From Natural Language Supervision. *arXiv preprint arXiv:2103.00020*.

- [3] Brown, T. B., et al. (2020). Language Models are Few-Shot Learners. *arXiv preprint arXiv:2005.14165*.
- [4] Driess, D., et al. (2023). PaLM-E: An Embodied Multimodal Language Model. *arXiv preprint arXiv:2303.03378*.
- [5] Kim, M. J., et al. (2024). OpenVLA: An Open-Source Vision-Language-Action Model. *arXiv preprint arXiv:2406.09246*.
- [6] Bai, J., et al. (2023). Qwen-VL: A Versatile Vision-Language Model for Understanding, Localization, and Generation. *arXiv preprint arXiv:2308.12966*.
- [7] DeepSeek. (2024). DeepSeek-V2: A Strong, Economical, and Efficient Mixture-of-Experts Language Model. Retrieved from <https://deepseek.com>